Project Title

Simple Intrusion Detection System (MiniIDS) for Log File Security Monitoring

Project Features

1. Log File Monitoring

Reads system log files (auth.log)

Checks each log line for suspicious behavior

2. Brute Force Detection

Detects repeated failed login attempts

Flags possible password guessing attacks

3. Blacklist IP Detection

Loads suspicious IP addresses from blacklist.txt

Detects if any blacklisted IP appears in logs

4. Alert System

Displays real-time alerts in the terminal

Saves alerts into alerts.txt

5. Event Counting

Counts:

Number of brute-force attempts
Number of blacklist detections

6. Basic Security Reporting

Shows summary report at the end of execution
Helps understand system security status

7. Colored Alert System

RED → High risk alerts (attacks)
YELLOW → log scanning activity
GREEN → normal login detection
Improves readability and security visualization

//FULL IMPLEMENTATION CODE

```
#include <iostream>
#include <fstream>
#include <string>
#include <ctime>
#include <unordered_set>
#include <windows.h>
using namespace std;
// color function
void setColor(int color) {
    SetConsoleTextAttribute(GetStdHandle(STD_OUTPUT_HANDLE), color);
}
// colors
#define RED 12
#define GREEN 10
#define YELLOW 14
#define WHITE 7
string getTime() {
    time_t now = time(0);
    return string(ctime(&now));
}
// counters
int bruteForceCount = 0;
```

```
int blacklistCount = 0;
// blacklist storage
unordered_set<string> blacklist;
// load blacklist
void loadBlacklist() {
    ifstream file("blacklist.txt");
    string ip;
    if (!file) {
        cout << "Warning: blacklist.txt not found!\n";
        return;
    }
    while (file >> ip) {
        blacklist.insert(ip);
    }

    file.close();
}

// alert system
void alert(string message, string level, int color) {
    setColor(color);
    cout << "[" << level << " ALERT] " << message << endl;
    setColor(WHITE);
    ofstream file("alerts.txt", ios::app);
    file << getTime() << " [" << level << "] " << message << endl;
```

```
    file.close();
}

int main() {
    cout << "   SIMPLE MINI IDS STARTED   \n";
    loadBlacklist();
    ifstream logFile("auth.log");
    string line;

    if (!logFile) {
        setColor(RED);
        cout << "Error: Cannot open auth.log file!" << endl;
        setColor(WHITE);
        return 1;
    }
    while (getline(logFile, line)) {
        setColor(YELLOW);
        cout << "Checking log: " << line << endl;
        setColor(WHITE);
        // brute force detection
        if (line.find("Failed password") != string::npos) {
            bruteForceCount++;
            alert("Repeated failed login attempt detected", "HIGH", RED);
        }
        // blacklist detection
```

```
        for (const auto& ip : blacklist) {
            if (line.find(ip) != string::npos) {
                blacklistCount++;
                alert("Blacklisted IP detected: " + ip, "HIGH", RED);
            }
        }

        // normal login detection
        if (line.find("login") != string::npos &&
            line.find("success") != string::npos) {
            setColor(GREEN);
            cout << "[INFO] Normal login detected" << endl;
            setColor(WHITE);
        }
    }
    logFile.close();
    cout << "      SCAN SUMMARY REPORT     \n";
    setColor(RED);
    cout << "Brute force events: " << bruteForceCount << endl;
    cout << "Blacklisted IP events: " << blacklistCount << endl;
    setColor(WHITE);
    cout << "\nReport saved in alerts.txt\n";

    return 0;
}
```